NLP4EMAI

Natural Language Processing for EMAI

Session 2. Text as Data: Processing and Normalization

Session outline

Where we are in the pipeline

Text as unstructured data

Tokenization

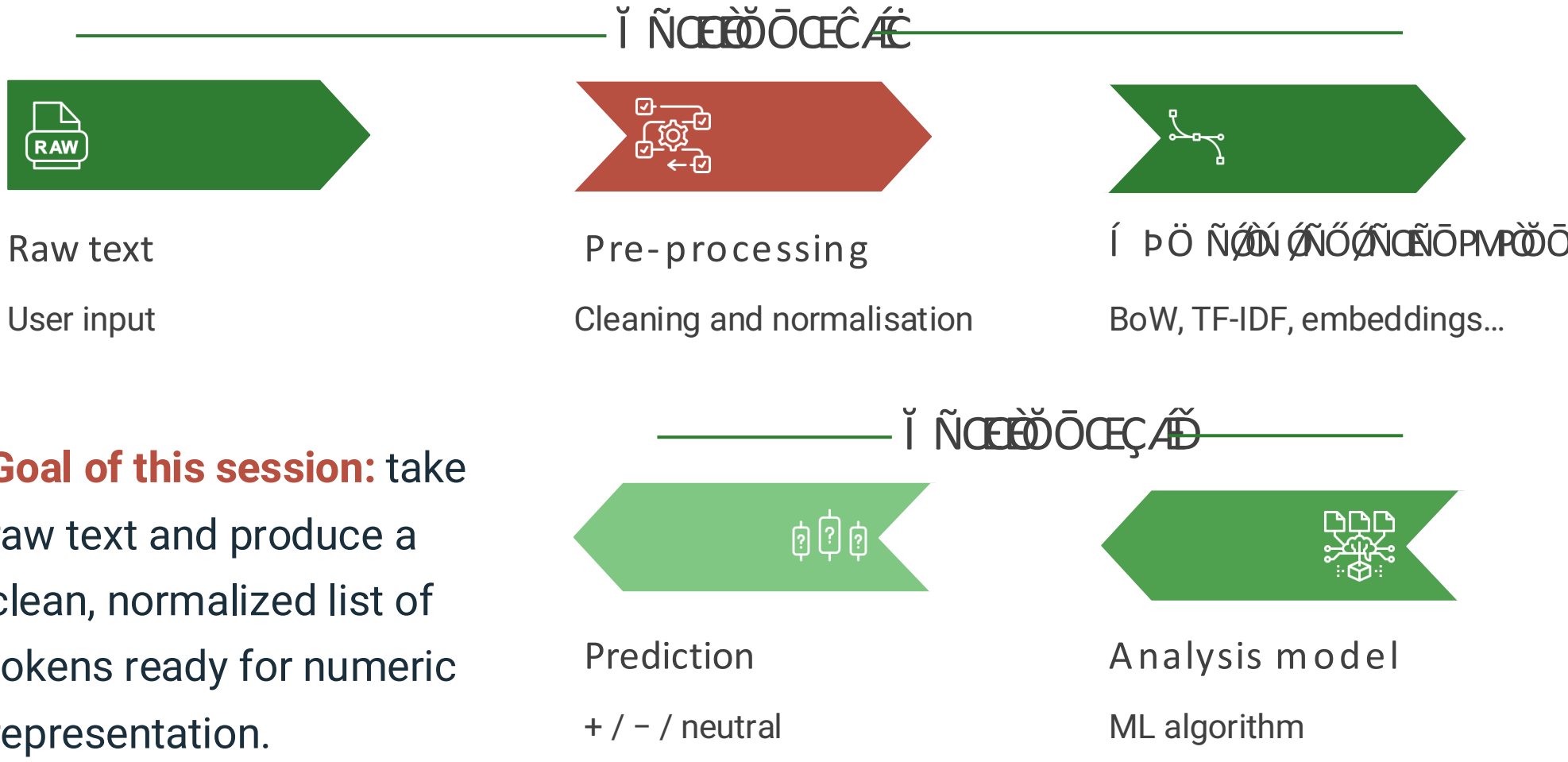
Normalization

Introducing Pandas

Activity 1: Hand-made preprocessing

Activity 2: Pandas preprocessing

Where we are in the pipeline



Goal of this session: take raw text and produce a clean, normalized list of tokens ready for numeric representation.

Session outline

Where we are in the pipeline

Text as unstructured data

Tokenization

Normalization

Introducing Pandas

Activity 1: Hand-made preprocessing

Activity 2: Pandas preprocessing

Text as unstructured data

Why can’t we feed text directly to a model?

Variability

> *”Great!” vs. “gr8!” vs. “AMAZING”*

Same meaning, completely different strings

Text as unstructured data

Why can’t we feed text directly to a model?

Ambiguity

> *”I love this place”*

Sarcasm? Genuine? The context changes everything

Text as unstructured data

Why can’t we feed text directly to a model?

Noise

> 🙄 **-- <p> -- XD – eigth**

Typos, HTML tags, URLs, emojis, punctuation scattered everywhere

Text as unstructured data

Why can’t we feed text directly to a model?

No fixed structure

> *”Blablablablabla… blablablabla….bla”*

Unlike a spreadsheet, text has no colums, no types, no fixed length

Text as unstructured data

Why can't we feed text directly to a model?

Variability Ambiguity Noise No fixed structure

Numerical models need fixed-size, clean, consistent inputs, and the text provides none of that out of the box.

Session outline

Where we are in the pipeline

Text as unstructured data

Tokenization

Normalization

Introducing Pandas

Activity 1: Hand-made preprocessing

Activity 2: Pandas preprocessing

Tokenization

What is a token?

Definition

Minimum unit of text that an NLP system uses as its basic element of analysis.

It is anything the system decides to treat as a meaningful piece of language

The model does not learn from raw text; it learns from tokens. Tokenization decisions shape what model can see.

Word

good

Most common token type

Emoji

☺ *:-D* *xD*

Emotionally significant

Ĩ Þ ŌŃÞ MÞŒŌ

! . ...

Sometimes meaningful, sometimes noise

Subword

Un- / -ing

Used in modern tokenizers

Tokenization

Tokenization types

1 Naïve split

text.split(“ “)

`“hello,“ → [‘hello,‘]`

Comma attached to word (punctuation is noise inside tokens)

2 Clean then split

Lowercase + remove punct + split

`“hello, JD! :)” →[‘hello’,’JD’]`

Cleaner tokens, but you lose emotionally relevant symbols like ‘:)’

3 Char-by-char

Build token character by character

`“: )” kept as one token`

Most flexible (design rules decide what counts as a token)

Regex-based defines precise patterns to identify tokens (e.g. dates, hashtags, URLs). More control, but requires manual rule design

Subword tokenization are algorithms like **BPE** or **WordPiece** split rare words into meaningful fragments. Used by all modern LLMs.

Session outline

Where we are in the pipeline

Text as unstructured data

Tokenization

Normalization

Introducing Pandas

Activity 1: Hand-made preprocessing

Activity 2: Pandas preprocessing

Normalization

What is normalization?

Definition

Normalization is the process of transforming raw text into a consistent, clean form so that equivalent expressions are treated identically by the model.

movie

great

loved

:D

“The Movie was GREAT!!! :D Loved it <3”

`[”movie”, “great”, ”loved”, “:D”]`

Before normalization

After normalization

Normalization

Reducing noise while preserving meaning

Lowercasing

"Movie" and "movie" → same token

Reduces vocabulary size (simple but effective).

Remove punctuation

"Hello!" → "hello"

Take care with *emojis*

Stopwords removal

"the", "a", "is" → removed

Frequent words with low semantic content.

What happens with "not"?

Further options

Numbers, special chars, HTML tags, …

Each choice is a design decision that affects what the model can learn.

Session outline

Where we are in the pipeline

Text as unstructured data

Tokenization

Normalization

Introducing Pandas

Activity 1: Hand-made preprocessing

Activity 2: Pandas preprocessing

Activity 1 - Hand-made preprocessing

From raw text to tokens

1 Text as strings

`Lowercase + remove punctuation.` First contact with **string manipulation**.

2 Naïve tokenization

Use `.split()` to discover **its limits** and what gets lost.

3 Char-by-Char tokenizer

Build a tokenizer from scratch. Preserve emoticons as tokens.

4 Complete pipeline

Integrate everything. Make and justify design decisions.

Activity 1 - Hand-made preprocessing

Text as strings: your first preprocessing steps

1 Text as strings

`Lowercase + remove punctuation.` First contact with **string manipulation**.

Goal

Manipulate raw text using only Python string methods, no libraries.

TODO 1 · Lowercase

```
lower_sentence = sentence.lower()
```

Why does this matter? Count how many unique forms

"Battery" - "battery" - "BATTERY" would create.

TODO 2 · Remove punctuation

```
if char.isalpha() or char.isdigit()
   or char == " ":
```

What happens to ":)" and "..."?

Is losing them acceptable here?

TODO 3 · clean_text( ) function

```
def clean_text(text):
    result = []
    for char in text.lower():
        ...
```

Wrap the logic.

Test on 3 different sentences before moving to Activity 1.2.

Activity 1 - Hand-made preprocessing

Text as strings: your first preprocessing steps

2 Naïve tokenization

Use `.split()` to discover **its limits** and what gets lost.

Goal

Split text into tokens with `.split()`, and discover where it breaks.

TODO 4 · Tokenise before cleaning

```
raw_tokens = raw_sentence.split(" ")
```

Observe: "NLP..." "Python!" "It's" ":)"

Which are problems? Which might be useful?

TODO 5 · Tokenise after cleaning

```
clean_tokens =

  clean_text(raw_sentence).split(" ")
```

Tokens are cleaner, but what's MISSING?

Is that acceptable for emotion detection?

TODO 6 · split( ) vs split(" ")

```
# Double spaces after removing punct:

tokens = text.split()   # ← better
```

.split(" ") creates empty tokens.

.split() handles any whitespace, so use this.

Activity 1 - Hand-made preprocessing

Text as strings: your first preprocessing steps

3 Char-by-Char tokenizer

Build a tokenizer from scratch. Preserve emoticons as tokens.

Goal

Full control over tokenization by building a state machine that preserves emoticons.

TODO 7

```
tokenize_basic()
```

Implement the state machine above.

Drop punctuation silently. Save last buffer after loop.

TODO 8

```
tokenize_with_emoticons()
```

Check text[i:i+2] against EMOTICONS set.

If match: save buffer + append emoticon, skip 2 chars.

TODO 9

```
tokenize(keep_emoticons=True)
```

Wrap both versions in one function.

Bonus: why is this a good design pattern?

Activity 1 - Hand-made preprocessing

Text as strings: your first preprocessing steps

4 Complete pipeline

Integrate everything. Make and justify design decisions.

Goal

Integrate everything from 1.1 to 1.3. Make and justify every design decision.

`raw text` (*input*) → `lowercase_14()` (*TODO 10 (reuse)*) → `tokenise_14()` (*TODO 10 (reuse)*) → `remove_stopwords()` (*TODO 11*) → `remove_short_tokens()` (*TODO 12*) → `token list` (*output*)

TODO 10

```
tokenise_14(text,
keep_emoticons)
```

Adapt your char-by-char tokenizer.

Use EMOTICONS_14 (extended set).

TODO 11

```
remove_stopwords(tokens)
```

List comprehension: keep tokens

not in STOPWORDS. Watch out for 'not'!

TODO 12

```
remove_short_tokens(tokens)
```

Keep tokens with `len ≥ min_length`.

Always preserve emoticons.

TODO 13

```
pipeline(text, ...)
```

Call the four functions in order.

All parameters must flow through.

Session outline

Where we are in the pipeline

Text as unstructured data

Tokenization

Normalization

Introducing Pandas

Activity 1: Hand-made preprocessing

Activity 2: Pandas preprocessing

Introducing pandas

Scaling up: From one sentence to a whole dataset

Before pandas

- One sentence at a time
- Manual loops
- Hard to scale
- No easy way to load CSV files

Whit pandas

- Process a **whole CSV in one call**
- Apply **functions column-wide**
- Easy to **inspect and debug**
- Same logic, industrial scale

`pd.read_csv()`	Load a dataset from a CSV file into a DataFrame
`df["text"].apply(fn)`	Apply your *clean* function to every row
`df["tokens"] = ...`	Store the result as a new column

Session outline

Where we are in the pipeline

Text as unstructured data

Tokenization

Normalization

Introducing Pandas

Activity 1: Hand-made preprocessing

Activity 2: Pandas preprocessing

Activity 2 – Pandas preprocessing

Scaling up with pandas

B Load dataset

```
pd.read_csv()
```

Inspect shape, labels, missing values.

C Apply pipeline

```
df['tokens'] =
df['text’]
  .apply(pipeline)
```

Inspect shape, labels, missing values.

D Token stats

Mean/min/max tokens.

Longest review.

Top 20 tokens.

E +/- Vocabulary

Compare most frequent tokens in positive vs negative reviews.

F Reflection

Discuss results.

Design decisions.

Session 3 preview.

Activity 1 - Hand-made preprocessing

Text as strings: your first preprocessing steps

B Load dataset

TODO 1

```
df = pd
   .read_csv(
      "reviews.csv”
    )
```

Two columns:

'text' and 'sentiment'.

TODO 2

```
df.shape  df.head()

df['sentiment’]
  .value_counts()

df.isnull().sum()
```

Are classes balanced?
Any missing values?

C Apply pipeline

TODO 3 and 4

```
# TODO 3
df["tokens"] = df["text"].apply(pipeline)
# TODO 4 — inspect with .iterrows()
```

.apply(fn) calls pipeline() on every row

No loop needed.

Expected output after Part C

[positive] The battery life on this phone is absolutely amazing...

→ `['battery', 'life', 'phone', 'absolutely', 'amazing']`

Activity 1 - Hand-made preprocessing

Text as strings: your first preprocessing steps

D Token stats

TODO 5

```
df['n_tokens'] =

df['tokens'].apply(len)
```

Print mean / min / max.

Are there very short reviews?

TODO 6

```
idx =
df['n_tokens'].idxmax()

df.loc[idx, 'text']
```

Find the review with the most tokens.

TODO 7

```
all_tokens =
  [t for tl in
   df['tokens'] for t in
  tl]

Counter(all_tokens)
  .most_common(20)
```

Flatten all lists.

Top 20 across dataset.

E +/- Vocabulary

TODO 8

```
pos = [t for tl, s in
        zip(df["tokens"],
        df["sentiment"])
             for t in tl
                 if s == "positive"]

Counter(pos).most_common(10)
```

Same pattern for 'negative'. Compare the two top-10 lists.

Look for tokens that appear in BOTH.

What to look in Part E (+/- Vocabulary)

POSITIVE top tokens → `love · amazing · best · brilliant · great`

NEGATIVE top tokens → `terrible · awful · waste · horrible · never`

Activity 1 - Hand-made preprocessing

Text as strings: your first preprocessing steps

F Reflection

Q1 – Tokens in both top-10 lists?	Q2 – Emoticons in top tokens?	Q3 – Add words to STOPWORDS?	Q4 – What is the next step?
What does it mean for a classifier if a token appears equally often in positive AND negative reviews?	Run the pipeline with `keep_emoticons=False`. Does it change the top-10?	Would removing frequent tokens lose sentiment information? Always check context first.	We have token lists. How do we turn them into numbers a model can learn from?

What have we learned in this session?

- **Raw text cannot go directly into a model**, it must be converted into clean, structured tokens.
- **Preprocessing = design.** Lowercasing, punctuation removal and stopwords all involve trade-offs.
- **Tokens are not words**, they are whatever the system treats as a meaningful unit (including emojis!).
- Pandas lets us apply the same pipeline to an entire dataset efficiently and reproducibly.
- **Every choice** (keep "not"? keep ":)"?) **directly affects** what the model can learn about sentiment.

NLP4EMAI

Natural Language Processing for EMAI

Session 2. Text as Data: Processing and Normalization